

# Political Market AI — Master Build Document

## Project objective

Build a political-market event intelligence system that converts geopolitical and political developments into structured market signals. The system should detect events, estimate how surprising and durable they are, map them to affected assets, compare them to historical analogs, and produce actionable market interpretations for short-term and medium-term time horizons.

## Core design principle

The system should not ask, “Was this headline bullish or bearish?” It should ask:

- What happened?
- Who said it?
- Is it credible?
- How unexpected is it?
- How persistent is it?
- Which transmission channels matter?
- Which assets are exposed?
- How did similar events affect markets?
- What is the likely reaction path over minutes, hours, days, and a week?

## Full system architecture

### 1. Ingestion layer

Collect raw inputs from political, market, and contextual sources.

Inputs:

- Political text: speeches, official statements, ministry releases, Truth Social posts, White House releases, sanctions notices, ceasefire announcements, military or maritime notices
- Market data: crude futures, energy ETFs, SPY, QQQ, airlines, defense stocks, shipping proxies, Treasury yields, VIX, FX, options-implied volatility
- Context data: prior similar events, chokepoint dependency, shipping route relevance, energy sensitivity by region, macro backdrop, consensus expectations when available

Output:

- timestamped raw event packets
- synchronized market snapshots around each event

## 2. Event extraction layer

Turn messy language into structured machine-readable events.

Fields to extract:

- actor
- target
- action
- location
- time reference
- source channel
- policy type
- coercion level
- military escalation signal
- supply-chain relevance
- commodity relevance
- implementation status
- reversibility
- caveats / conditions
- contradiction markers

## 3. Event classification layer

Assign each event to one or more market-relevant categories.

Top-level classes:

- blockade
- reopening
- ceasefire
- sanctions
- tariff threat
- military strike
- election result
- treaty / negotiation progress
- export restriction
- diplomatic breakdown
- shipping disruption
- energy supply relief

Sub-labels:

- de-escalation
- conditional de-escalation
- escalation
- signaling without implementation
- implementation-confirmed

- rumor / weak confirmation

#### **4. Scoring engine**

Convert the event into interpretable risk dimensions.

Core scores:

- surprise score
- credibility score
- persistence score
- transmission score
- conflict score
- fragility score
- market salience score

Interpretation:

- high surprise + high transmission = potential immediate move
- low persistence + high conflict = fade risk
- low credibility = no-trade or monitor

#### **5. Historical similarity engine**

Retrieve prior events with similar actors, event types, channels, and contradiction patterns.

Similarity dimensions:

- same geography
- same actors
- same asset channel
- same event class
- same contradiction pattern
- same persistence profile
- same macro regime

#### **6. Event-study engine**

Measure abnormal returns around event windows.

Windows:

- 0–15 minutes
- 15 minutes–2 hours
- event to close
- next day
- next week

Method:

- estimate expected baseline return from recent market relationships
- compare realized move to expected move
- compute abnormal return and abnormal volatility

## 7. Market mapper

Translate event characteristics into exposed assets and sectors.

Examples:

- Strait reopening → crude down initially, airlines up, broad equities relieved, inflation-sensitive assets supported
- blockade remains / military caveat → oil downside limited, defense supported, VIX risk remains, reversal risk elevated
- sanctions escalation → energy, shipping, EM FX, inflation-sensitive assets

## 8. Forecast / scenario engine

Generate scenario-based expectations.

Time horizons:

- next 15 minutes
- next hour
- next trading day
- next week

Outputs:

- base case
- upside case
- downside case
- invalidation triggers
- persistence vs fade conditions

## 9. Decision layer

Convert intelligence into usable analyst or trading outputs.

Decision states:

- act
- cautious act
- monitor
- no-trade
- invalid / unresolved

## Recommended build phases

### Phase 1 — Schema and storage

Build:

- event schema
- score definitions
- asset mapping tables
- historical event table
- market snapshot schema

Deliverables:

- JSON schema
- SQLite/Postgres schema
- label taxonomy

### Phase 2 — Ingestion

Build connectors for:

- official feeds / scraped statements
- manual test inputs
- market data feeds
- context datasets

### Phase 3 — Extraction + classification

Build:

- rule-based extractor first
- LLM-assisted parser second
- taxonomy-based classifier
- contradiction/caveat detector

### Phase 4 — Scoring

Build deterministic first-pass scoring functions for:

- surprise
- credibility
- persistence
- transmission
- conflict
- fragility

## **Phase 5 — Historical analog engine**

Build:

- similarity search
- analog retrieval table
- prior reaction summary logic

## **Phase 6 — Event-study engine**

Build:

- expected return baseline
- event window selection
- abnormal return and abnormal volatility calculation

## **Phase 7 — Reporting**

Build:

- event summary card
- asset bias card
- scenario card
- alert message format

## **Phase 8 — Execution interface**

Only after intelligence is validated:

- expose signal API
- push trade / monitor / no-trade outputs
- integrate with downstream bot logic

## **One-page strategy summary**

Build an AI system that converts political and geopolitical events into structured market intelligence. The system should detect events, classify them, score their importance, map them to affected assets, compare them to historical analogs, and estimate likely short-term and medium-term market reactions.

Markets do not react to headlines alone. They react to the combination of surprise, credibility, persistence, and transmission into fundamentals. A statement like “the Strait is reopened” may be bullish for risk and bearish for crude, but if it is conditional or contradicted by “the blockade remains,” the system must detect that fragility and raise reversal risk.

MVP focus:

- Strait of Hormuz

- sanctions affecting oil flow
- naval blockades
- ceasefires affecting shipping
- OPEC / supply disruption headlines

MVP assets:

- Brent, WTI, USO
- XLE
- JETS
- SPY, QQQ
- VIX
- defense names

Core scores:

- surprise
- credibility
- persistence
- transmission
- conflict / contradiction

For each event, produce:

- structured event summary
- key scores
- affected assets
- predicted near-term bias
- reversal risk
- historical analogs
- confidence level

Key rule: Do not let the system trade dramatic language alone. It should act only when the event is credible, relevant, and likely to persist long enough to affect market expectations.

## **AI analyzer spec**

### **Role**

You are a geopolitical-market event intelligence analyzer. Your job is to convert political or geopolitical developments into structured market intelligence. You are not a generic sentiment model. You must evaluate event surprise, credibility, persistence, transmission channels, contradiction risk, and likely asset impact.

## Mission

Given one or more political events, produce: 1. structured event extraction 2. event classification 3. score estimates 4. affected assets and likely direction 5. short-term and medium-term scenario analysis 6. reversal and fragility assessment 7. final action state: act / cautious act / monitor / no-trade

## Output structure

```
{
  "event_summary": "",
  "event_type": "",
  "subtype": "",
  "actors": [],
  "location": "",
  "targets": [],
  "implementation_status": "",
  "scores": {
    "surprise": 0,
    "credibility": 0,
    "persistence": 0,
    "transmission": 0,
    "conflict": 0,
    "fragility": 0
  },
  "affected_assets": [
    {
      "asset": "",
      "bias": "",
      "confidence": ""
    }
  ],
  "time_horizon_view": {
    "next_15m": "",
    "next_1d": "",
    "next_1w": ""
  },
  "historical_analog_logic": "",
  "reversal_risk": "",
  "action_state": "",
  "confidence": "",
  "notes": ""
}
```

## Score definitions

- surprise: how unexpected relative to prior expectations

- credibility: authority and practical ability to implement claim
- persistence: likely duration of effect
- transmission: directness of impact on oil, inflation, trade, earnings, or funding conditions
- conflict: contradiction or ambiguity inside the event bundle
- fragility: chance the initial interpretation fails quickly

## Interpretation rules

- “reopened during ceasefire” is not equal to “durably reopened”
- “blockade remains” reduces persistence of relief
- direct oil-flow relevance increases transmission score
- military or diplomatic caveats increase fragility
- contradictory signals should reduce confidence and raise reversal risk

## Action-state policy

- act: only if signal is credible, high-transmission, sufficiently persistent, and not internally contradictory
- cautious act: if the event is meaningful but contains fragility
- monitor: if event matters but implementation or persistence is uncertain
- no-trade: if event is mostly rhetoric, low transmission, or highly contradictory

## Python project skeleton

### Structure

```
political_market_ai/
|
|— app/
|   |— config.py
|   |— main.py
|   |— ingestion/
|   |— extraction/
|   |— classification/
|   |— scoring/
|   |— history/
|   |— event_study/
|   |— mapping/
|   |— forecast/
|   |— decision/
|   |— reporting/
|   |— models/
|
|— data/
|   |— raw/
|   |— processed/
|   |— historical_events/
```

```
|   └─ market_snapshots/
|
|─ tests/
|─ notebooks/
|─ requirements.txt
└─ README.md
```

## Minimal starter modules

- config.py
- models/schemas.py
- extraction/event\_parser.py
- classification/event\_classifier.py
- scoring/transmission.py
- mapping/asset\_mapper.py
- decision/decision\_engine.py
- main.py

## Trading bot logic block

The trading bot is not the core intelligence layer. It is the execution layer on top of the analyzer.

### Bot should only consider trades when:

- surprise is high enough
- credibility is high enough
- transmission is high enough
- conflict is not too high
- action\_state is act or cautious\_act
- price action confirms the thesis

### Trade classes

- Class A: high surprise, high credibility, high transmission, low conflict
- Class B: high transmission but conditional persistence
- Class C: contradiction too high, watch only

### Bot outputs

- no\_trade
- watch\_only
- trade\_ready
- reduce\_risk
- flatten\_now

## Final rule

The analyzer creates the thesis. The market confirms the thesis. The bot executes only after both align.

## First practical coding step

Do not start with the trading bot.

Start by building the schema and first pipeline shell.

### Step 1

Create the project folder and the 8 core starter files:

- `app/config.py`
- `app/models/schemas.py`
- `app/extraction/event_parser.py`
- `app/classification/event_classifier.py`
- `app/scoring/transmission.py`
- `app/mapping/asset_mapper.py`
- `app/decision/decision_engine.py`
- `app/main.py`

### Immediate goal of step 1

Be able to input one raw event headline and get back:

- event summary
- event type
- subtype
- transmission score
- mapped assets
- action state

That gives you a working vertical slice before you add historical analogs, event studies, or execution.

### Definition of done for step 1

You can run one command and see a structured event object printed from a sample Hormuz-style headline.

That is the correct place to start making this a full codebase.